블록체인(01) — 중간고사 대비 100제 (3)

범위: 0. Introduction ~ 9. 솔리디티 (1~8주차) · 기존 100제 두 세트와 겹치지 않게 시나리오·계산·과제 코드 위주로 새로 출제

구성 (실제 시험 형식: O/X · 단답형 · 4지선다 · 단순기술형 · 복합기술형)

- Ⅰ. O/X — 1~25번 (25문항)
- Ⅱ. 4지선다 — 26~55번 (30문항)
- Ⅲ. 단답형·계산 — 56~80번 (25문항)
- Ⅳ. 단순기술형 — 81~92번 (12문항)
- Ⅴ. 복합기술형 — 93~100번 (8문항)
- 문항 앞 [ ]는 단원. 정답 및 해설은 맨 뒤에 있음. 계산 문제는 모두 코드로 검산했고, SavingWallet 문항은 실제 컴파일·실행으로 확인함

Ⅰ __ / 25	Ⅱ __ / 30	Ⅲ __ / 25	Ⅳ __ / 12	Ⅴ __ / 8	합계 __ / 100

Ⅰ. O/X

맞으면 O, 틀리면 X

1. [탄생] 브레튼우즈 협정(1944)에서는 45개국 대표가 모여 미국 달러를 기축통화로 하고 달러를 금에 고정하는 국제 통화 질서를 만들었다.

2. [탄생] 1·2차 세계대전 중 참전국들은 전쟁 자금을 마련하기 위해 금본위제를 일시적으로 폐지했다.

3. [탄생] 루스벨트의 뉴딜 정책(1933)은 금본위제를 강화하고 통화량을 줄인 긴축 정책이었다.

4. [탄생] 전자화폐는 정부·금융기관이 현금이나 예금으로의 교환을 보장하지만, 가상화폐는 플랫폼 운영사가 발행해 정부의 통제를 받지 않는다.

5. [탄생] eCash는 은닉 서명을 사용했으므로 중앙 기관 없이 이중 지불을 검증한 최초의 암호화폐이다.

6. [기초] 하나의 지갑에서는 하나의 계정만 만들 수 있다.

7. [기초] 합의를 거쳐 동기화된 블록은 더 이상 위·변조가 불가능하다.

8. [기초] 비허가형 블록체인에서는 합의 알고리즘이 보안과 신뢰를 전적으로 담당한다.

9. [기초] 컨소시엄 블록체인은 단일 기관에 권한이 집중되어 있으므로 참여자 간 이해 조율이 필요 없다.

10. [암호] 8바이트 패스워드와 5TB짜리 USB 전체 파일을 같은 해시 함수에 넣으면 두 해시 값의 길이는 같다.

11. [암호] BLAKE는 SHA-3 공모에서 최종 선정된 알고리즘이며, BLAKE2는 RFC 7693으로 표준화되었다.

12. [암호] 회로로 표현할 때 Poseidon은 SHA-256보다 곱셈 게이트 수가 훨씬 적다.

13. [암호] 집계 서명은 단일 서명 하나만 놓고 보면 ECDSA·RSA보다 계산 비용이 작다.

14. [암호] EdDSA는 RSA·ECDSA보다 오래전에 표준화되어 안전성 검증 기간이 길다.

15. [암호] 전자서명으로 '누가 서명했는지'를 확인하려면 실제 신원과 서명키의 연결을 보증하는 수단(예: PKI 인증서)이 필요하다.

16. [합의] 생존성(Liveness)은 정직한 노드들이 서로 모순되는 거래 이력을 확정하지 않는 성질이다.

17. [합의] 하드포크·소프트포크는 운영 규칙이 바뀌어 생기는 분기로, 대표 블록이 동시에 생겨 장부가 갈라지는 체인 분기와는 다른 개념이다.

18. [합의] PBFT는 참여자가 늘어나도 메시지 양이 거의 일정하므로 개방형 대규모 네트워크에 적합하다.

19. [합의] PoS에서 검증 위원은 지분이 많은 노드부터 순서대로 확정적으로 배정된다.

20. [비트코인] 블록 해시를 계산할 때 블록 바디의 트랜잭션 카운터도 해시 입력에 포함된다.

21. [솔리디티] delegatecall은 다른 컨트랙트의 코드와 저장소를 모두 빌려 쓴다.

22. [비트코인] 풀 노드는 매 블록마다 nBits를 새로 계산해 채굴자에게 알려 준다.

23. [비트코인] 코인베이스 트랜잭션도 머클 루트 계산에 포함된다.

24. [이더리움 기초] 이더리움도 비트코인처럼 2016블록마다 난이도를 조절한다.

25. [이더리움] 아카이브 노드는 과거 상태를 삭제(프루닝)해 저장 공간을 절약한다.

Ⅱ. 4지선다

가장 알맞은 것을 고르시오

26. [탄생] 강의에서 금의 특성을 화폐의 요건과 연결했다. 연결이 **틀린** 것은?

① 변하지 않는 화려한 색상 — 내재적 가치
② 제한된 매장량, 어려운 채굴 — 희소성
③ 상하거나 변하지 않음 — 보편성
④ 전 세계 모든 사람이 인정 — 보편성

답 _____

27. [탄생] 사토시 나카모토가 P2P Foundation 게시글에서 기존 화폐 시스템을 비판한 내용이 **아닌** 것은?

① 중앙은행이 화폐 가치에 대한 신뢰를 저버렸다
② 시중은행이 일부만 유보금으로 남기고 대출해 신용 거품을 만든다
③ 은행의 과도한 비용 때문에 소액결제가 어렵다
④ 금 채굴에 너무 많은 에너지가 들어 화폐 발행이 어렵다

답 _____

28. [탄생] 사토시 나카모토에 대한 강의 내용으로 옳지 **않은** 것은?

① 현재까지 누군지 밝혀지지 않았다
② 개인이 아닌 단체일 가능성도 있다
③ 본인은 1975년생 일본인이라고 주장했다
④ 2009년 Java 기반 프로토타입 시스템을 공개했다

답 _____

29. [기초] 블록체인의 트랜잭션(상태를 변화시키는 기록)에 해당하지 **않는** 것은?

① 화폐 전송
② 스마트 계약 함수 호출
③ 블록 생성
④ 노드가 이전에 연결한 Peer의 IP 주소를 캐시에 저장

답 _____

30. [기초] DeFi에 대한 설명으로 옳지 **않은** 것은?

① 모든 거래가 블록체인에 기록되어 투명하다
② 은행 계좌가 없는 사람도 참여할 수 있다
③ 스마트 계약 버그, 보안 사고, 규제 불확실성이 한계이다
④ 중앙 기관이 예금자 보호를 보장하므로 원금 손실 위험이 없다

답 _____

31. [기초] 개방형(Public) 블록체인의 특징으로 옳지 **않은** 것은?

① 특정 기관의 검열·통제가 불가능하다
② 누구나 참여해 트랜잭션 내용을 확인할 수 있다
③ 초당 처리량(TPS)이 높아 확장성이 뛰어나다
④ 완전 비허가형 구조이다

답 _____

32. [암호] 해시 값이 데이터의 '지문' 역할을 할 수 있는 근거로 강의에서 든 것이 **아닌** 것은?

① 보안 성질(역상·제2역상·충돌 저항성)　② 고정 길이 출력

③ 눈사태 효과　④ 비밀키를 아는 사람만 계산할 수 있음

답 _____

33. [암호] Tx1~Tx8로 만든 머클 트리(해시1~8 → 해시12·34·56·78 → 해시1234·5678 → 루트해시)에서 **Tx6**의 머클 증명은?

① {해시5, 해시78, 해시1234, 루트해시}

② {해시6, 해시56, 해시5678, 루트해시}

③ {해시5, 해시7, 해시8, 루트해시}

④ {해시56, 해시1234, 루트해시}

답 _____

34. [암호] RSA 서명($\sigma = m^d \bmod n$)의 안전성이 깨지는 경우는?

① 타원곡선 위 점의 덧셈이 쉬워질 때　② 큰 수 n의 소인수분해가 쉬워질 때

③ 해시 함수의 눈사태 효과가 사라질 때　④ 난수 생성기가 고장 났을 때

답 _____

35. [암호] 영지식 증명의 활용으로 보기 **어려운** 것은?

① 생년월일을 공개하지 않고 성인임을 증명

② 여권 번호를 공개하지 않고 특정 국가 시민임을 증명

③ L2 롤업에서 수천 개 트랜잭션의 유효성을 간결하게 증명

④ 모든 노드가 거래 원문을 내려받아 하나씩 재실행해 검증

답 _____

36. [암호] 공격자가 Alice의 계약서 x를 손에 넣고, 해시 값이 같은 **다른** 위조 계약서 x′를 만들려 한다. 이를 막는 성질은?

① 역상 저항성　② 제2역상 저항성

③ 충돌 저항성　④ 눈사태 효과

답 _____

37. [합의] 일부 노드가 네트워크에서 끊겼는데도 나머지 노드들이 블록 생성과 합의를 멈추지 않고 이어 간다. 이와 가장 관련된 합의의 핵심 요소는?

① 안전성　② 생존성

③ 최종성　④ 익명성

답 _____

38. [합의] 비트코인 최장 체인 규칙에서 '가장 긴 체인'의 실제 판단 기준은?

① 블록 개수　② 포함된 트랜잭션 수

③ 누적 작업량　④ 가장 먼저 도착한 체인

답 _____

39. [합의] PoS 검증자의 동작 과정을 순서대로 바르게 나열한 것은?

(가) 블록 승인 시 검증 위원에게 보상 (나) 자산을 예치해 검증자로 등록 (다) 블록이 제안되면 검증 위원이 검증하고 투표 (라) 규칙에 따라 무작위로 검증 위원(제안자 포함) 배정

① (나) → (라) → (다) → (가)
② (라) → (나) → (다) → (가)
③ (나) → (다) → (라) → (가)
④ (다) → (나) → (라) → (가)

답 _____

40. [합의] DPoS에 대한 설명으로 옳지 **않은** 것은?

① 지분을 가진 일반 노드가 대리투표로 의사결정에 참여한다
② 상위 득표 후보 여럿이 블록 생산자로 선정된다
③ 규칙을 어긴 대표자는 투표로 교체될 수 있다
④ 대표 집단을 확률적(무작위)으로 선정한다

답 _____

41. [합의] PBFT 단계 중 '각 노드가 블록을 검증한 뒤 자신의 판단을 모든 검증 노드에게 전파'하는 단계는?

① Pre-Prepare
② Prepare
③ Commit
④ Reply

답 _____

42. [비트코인] 비트코인 주소에 대한 설명으로 옳지 **않은** 것은?

① 지갑(공개키)에서 파생된 값이다
② 비트코인에서는 화폐의 잠금·해제 조건 라벨 역할을 한다
③ 하나의 지갑에서 여러 주소를 만들 수 있다
④ 주소만 알면 공개키와 개인키를 쉽게 역산할 수 있다

답 _____

43. [비트코인] 블록 헤더의 '버전' 필드가 바뀌는 대표적인 경우는?

① 블록에 트랜잭션이 추가될 때
② 소프트포크처럼 프로토콜 기능이 변할 때
③ 넌스를 다시 고를 때
④ 난이도가 재조정될 때

답 _____

44. [DApp] Web3.js / Ethers.js의 역할로 옳은 것은?

① 로컬에서 풀 노드를 구동한다
② JavaScript 코드로 스마트 계약 호출, 지갑 연결, 트랜잭션 전송을 한다
③ Solidity 코드를 EVM 바이트코드로 컴파일한다
④ 테스트용 ETH를 무료로 지급한다

답 _____

45. [비트코인] 트랜잭션 수수료에 대한 설명으로 옳지 **않은** 것은?

① 모든 입력 합에서 모든 출력 합을 뺀 차액이다

② 지갑이 데이터 크기·네트워크 상황을 분석해 권장 수수료를 계산한다

③ 사용자가 직접 선택·조정할 수 있다

④ 출력 필드에 '수수료' 항목을 따로 만들어 채굴자 주소로 보내야 한다

답 _____

46. [비트코인] 비트코인 네트워크 동작을 순서대로 나열한 것은?
(가) 후보 블록 구성 후 넌스 탐색 (나) 지갑에서 트랜잭션 생성·브로드캐스트 (다) 수신 노드가 검증 후 MemPool에 저장
(라) 블록을 받은 노드가 검증 후 자신의 시퀀스에 연결 (마) 작업 증명을 마친 블록 전파

① (나) → (다) → (가) → (마) → (라)

② (나) → (가) → (다) → (마) → (라)

③ (다) → (나) → (가) → (라) → (마)

④ (나) → (다) → (마) → (가) → (라)

답 _____

47. [비트코인] nBits = 0x1b0404cb일 때, 32바이트 타겟 T를 16진수로 쓰면 C(0404cb) **앞**과 **뒤**에 오는 0 바이트 수는?

① 앞 5바이트, 뒤 24바이트

② 앞 4바이트, 뒤 25바이트

③ 앞 5바이트, 뒤 27바이트

④ 앞 24바이트, 뒤 5바이트

답 _____

48. [비트코인] 최초 보조금이 50 BTC일 때 블록 높이 700,000의 블록 보조금은?

① 12.5 BTC

② 6.25 BTC

③ 3.125 BTC

④ 1.5625 BTC

답 _____

49. [이더리움 기초] 비트코인 초창기에 Zero Confirmation이 '리스크 대비 이익이 컸던' 이유는?

① RBF 기능이 아직 없었기 때문

② 소액 거래 위주였고 미확정 거래가 빠르게 블록에 포함되는 편이었기 때문

③ 51% 공격이 원천적으로 불가능했기 때문

④ 트랜잭션 수수료가 0이었기 때문

답 _____

50. [이더리움 기초] EVM에 대한 설명으로 옳지 **않은** 것은?

① 노드 기기에서 실행되는 바이트코드 실행 소프트웨어이다

② 모든 노드가 같은 입력에 대해 같은 출력을 얻게 한다

③ 어떤 하드웨어·OS를 쓰든 동일한 실행 환경을 보장한다

④ 노드의 OS에 따라 실행 결과가 조금씩 달라지는 것을 허용한다

답 _____

51. [이더리움 기초] 블록 용량 제한에 대한 강의 비교표의 내용으로 옳은 것은?

① 이더리움: 1MB 고정

② 비트코인(현재): 최대 4M Weight, 이더리움: Gas Limit 기반(30M~60M Gas)

③ 비트코인(현재): Gas Limit 기반

④ 둘 다 블록 용량 제한이 없다

답 _____

52. [이더리움] 이더리움의 상태 트리(state tree)와 storageRoot에 쓰이는 자료구조는?

① 이진 탐색 트리

② 머클-패트리샤 트리(MPT)

③ B+ 트리

④ 힙

답 _____

53. [이더리움] EIP-1559(Type-2)가 도입된 배경으로 옳은 것은?

① 레거시 방식은 수수료가 0이어서 스팸이 많았다

② 레거시 방식은 사용자가 Gas Price를 직접 정하는 경매 방식이라, 혼잡 시 과도한 수수료와 예측 어려움이 있었다

③ PoW에서 넌스를 없애기 위해서였다

④ 컨트랙트 생성 시 to 필드를 쓰기 위해서였다

답 _____

54. [이더리움] L2 롤업에 대한 설명으로 옳지 **않은** 것은?

① 메인넷(L1) 외부 네트워크에서 실행을 처리한다

② 트랜잭션을 모아 압축·증명한 뒤 L1에 관련 데이터를 게시한다

③ L1의 보안성과 L2의 처리 효율성을 함께 얻는다

④ L2는 L1과 완전히 독립된 체인이라 L1에 아무것도 기록하지 않는다

답 _____

55. [솔리디티] 함수 타입 중 '주소를 통해 호출'하는 유형은?

① 내부 타입(internal)

② 외부 타입(external, public)

③ view 타입

④ pure 타입

답 _____

Ⅲ. 단답형 · 계산

답과 간단한 풀이를 쓰시오

56. [탄생] 다음 사건의 연도를 쓰시오.
① 데이비드 차움의 은닉 서명 제안 ② DigiCash 설립 ③ 웨이 다이의 B-Money 제안 ④ 비트코인 C++ 프로토타입 공개

57. [탄생] 사이퍼펑크 운동의 주요 활동 두 가지를 쓰시오.

58. [기초] 비트코인 제네시스 블록 해시는 000000000019d6689c…처럼 0이 여러 개로 시작한다. 그 이유를 쓰시오.

59. [기초] 다음 상황에 가장 알맞은 블록체인 유형(Public / Private / Hybrid / Consortium)을 쓰시오.
① 한 은행의 내부 결제 시스템 ② 여러 은행이 함께 운영하는 무역 금융망 ③ 누구나 참여하는 암호화폐 ④ 공공 정보는 공개하고 민감 정보는 비공개로 함께 운영하는 행정 시스템

60. [암호] 트랜잭션이 5개인 비트코인 블록의 머클 트리를 만든다. ① 각 단계(리프 → 루트)에서 해시 노드 수가 어떻게 되는지 쓰고 ② Tx5의 포함을 증명하는 데 필요한 형제 해시 개수를 쓰시오.

61. [암호] 3-of-5 다중 서명 지갑이 있다. ① 정상 사용자가 키를 최대 몇 개까지 잃어버려도 서명할 수 있는가? ② 공격자는 최소 몇 개의 키를 훔쳐야 유효한 서명을 만들 수 있는가?

62. [암호] 각 상황과 관련된 전자서명의 성질을 쓰시오.
① 계약서 금액을 한 글자만 바꿔도 서명 검증이 실패한다 ② 서명자가 나중에 '나는 서명한 적 없다'고 주장할 수 없다 ③ 서명자만 아는 키로 서명했으므로 누가 서명했는지 확인할 수 있다

63. [합의] 노드 13개인 PBFT 네트워크가 있다. ① 견딜 수 있는 최대 악의적 노드 수 ② Commit 단계로 가기 위해 필요한 일치 메시지 수를 쓰시오.

64. [합의] 블록 A 뒤에 블록 B와 C가 동시에 생성되었다. 이후 B 뒤에 D, C 뒤에 E가 거의 동시에 붙었고, 잠시 뒤 E 뒤에 F가 연결되었다. 최종적으로 선택되는 체인과 스테일 블록을 쓰시오.

65. [합의] 해시 퍼즐의 조건이 '해시 값의 앞 16비트가 모두 0'이라면, 답을 찾기 위해 평균적으로 몇 번 시도해야 하는가? 찾은 답이 맞는지 검증하는 데는 해시를 몇 번 계산하면 되는가?

66. [비트코인] SPV 노드가 1년 동안 저장해야 하는 블록 헤더의 총 용량을 구하시오. (평균 10분에 1블록, 365일 기준)

67. [비트코인] 직전 2016개 블록이 만들어지는 데 12일이 걸렸다. ① 다음 주기의 난이도는 오르는가 내려가는가? ② 타겟 T는 대략 몇 배가 되는가?

68. [비트코인] 비트코인의 4번째 반감기는 2024년 블록 높이 840,000에서 일어났다. 다음(5번째) 반감기의 블록 높이와 그 이후의 블록 보조금을 쓰시오.

69. [비트코인] 다음 두 트랜잭션이 순서대로 확정되었다.
Tx1: A의 2.0 BTC UTXO 입력 → 출력 B 0.7 BTC, A 1.25 BTC
Tx2: B의 0.7 BTC, 0.5 BTC UTXO 입력 → 출력 C 1.1 BTC, B 0.08 BTC
① 두 트랜잭션의 수수료 합 ② 이 과정에서 파기된 UTXO ③ 최종적으로 남은(미사용) UTXO를 쓰시오.

70. [비트코인] 블록 높이 850,000 블록의 코인베이스 트랜잭션 출력이 3.2 BTC였다. 이 블록에 담긴 트랜잭션 수수료의 총합은?

71. [비트코인] ① 첫 번째 반감기 전(블록 0 ~ 209,999)까지 발행되는 비트코인은 몇 개인가? ② 이는 총 발행량의 몇 %인가?

72. [이더리움 기초] 비트코인에서 거래 확정을 위해 6 컨펌을 기다리면 평균 몇 분이 걸리는가? 컨펌 수를 늘릴수록 안전해지는 이유는?

73. [이더리움 기초] gas limit 80,000, gas price 25 gwei인 트랜잭션이 무한 루프로 가스를 모두 소진했다. ① 지불하는 수수료(gwei, ETH) ② 환불되는 가스 ③ 상태 변경은 어떻게 되는지 쓰시오.

74. [이더리움] EIP-1559 트랜잭션: Max Fee 25 gwei, Base Fee 22 gwei, Priority Fee 5 gwei, 사용 가스 21,000. 가스당 실제 지불 가격과 ① 총 수수료 ② 소각량 ③ 검증자 수령액을 구하시오.

75. [이더리움] Legacy 트랜잭션의 gasPrice 필드 값이 30,000,000,000이다. ① 이는 몇 gwei인가? ② 21,000 gas를 썼다면 수수료는 몇 ETH인가?

76. [이더리움] 같은 EOA가 똑같은 컨트랙트 코드를 두 번 배포했다. 두 CA의 주소는 같은가? 이유를 쓰시오.

77. [이더리움] 이더리움 PoS에서 하루(24시간) 동안의 슬롯 수와 에포크 수를 구하시오.

78. [DApp] Remix에서 `string public greet = "Hello";`로 선언한 뒤 배포하면, 코드에 함수를 작성하지 않았는데도 `greet` 버튼이 생긴다. 이 버튼의 정체와 public을 붙인 의도를 쓰시오.

79. [솔리디티] SavingWallet 과제에서 생성자에 _goal = 3(ETH 단위)이 들어왔다. goal에 wei 단위로 저장하는 코드 한 줄과 저장되는 값을 쓰시오.

80. [솔리디티] SavingWallet의 checkRemain()이 uint가 아니라 int를 반환하는 이유를 쓰시오. goal이 2 ETH이고 잔고가 2.5 ETH일 때 `goal - address(this).balance`를 uint로 바로 계산하면 어떻게 되는가?

Ⅳ. 단순기술형

핵심 개념을 2~4문장으로 서술하시오

81. [탄생] 1980년대 이후 기업에 의한 개인정보 수집이 늘어난 배경과 그 문제점을 쓰고, 이것이 블록체인 등장 배경과 어떻게 이어지는지 설명하시오.

82. [기초] 과거 블록 하나의 트랜잭션을 몰래 수정하면 어떤 일이 생기는지 '블록 해시', '이전 블록 해시', '머클 루트'를 넣어 설명하시오.

83. [암호] 배포 사이트에서 파일과 해시 값을 함께 내려받아 비교했는데 일치했다. 그래도 파일이 안전하다고 단정할 수 없는 경우를 쓰고, 전자서명이 이 문제를 어떻게 해결하는지 설명하시오.

84. [암호] 긴 메시지는 메시지 대신 해시 값에 서명한다(s = Sig_sk(h(m))). 그 이유와, 이때 해시 함수의 충돌 저항성이 깨지면 어떤 위험이 생기는지 설명하시오.

85. [합의] PoW에서 채굴자가 정직하게 행동하는 것이 '합리적'인 이유를 보상과 비용의 관점에서 설명하시오.

86. [합의] 최장 체인 규칙이 '확률적 최종성'만 제공하는 이유를 설명하고, PBFT의 최종성과 비교하시오.

87. [비트코인] SPV 노드는 블록 헤더만 가지고 있다. SPV 노드가 '내 거래가 블록에 포함되었는지'를 확인하는 방법을 머클 증명과 연결해 설명하시오.

88. [비트코인] 블록 타임스탬프의 두 가지 유효 조건이 각각 어떤 비정상적인 블록을 막으려는 것인지 설명하시오.

89. [비트코인] 채굴자가 MemPool에서 트랜잭션을 고르는 기준을 쓰고, 반감기로 블록 보조금이 계속 줄어들 때 이 기준이 네트워크 유지에 왜 중요해지는지 설명하시오.

90. [이더리움 기초] 스마트 컨트랙트가 배포 후 코드를 바꿀 수 없다(불변)는 성질의 장점과 단점을 하나씩 쓰시오.

91. [이더리움] 계정 필드 중 codeHash는 변경할 수 없지만 storageRoot는 계속 바뀐다. 그 이유를 각각 설명하시오.

92. [DApp] 같은 컨트랙트를 Remix VM에 배포할 때와 Sepolia(MetaMask 연결)에 배포할 때의 차이를 ① 연결되는 체인 ② 필요한 준비물 ③ 결과 확인 방법 관점에서 설명하시오.

Ⅴ. 복합기술형

조건에 맞게 단계별로 서술하시오

93. [비트코인] 블록 높이 880,000의 블록에 코인베이스 트랜잭션과 일반 트랜잭션 3개(수수료 0.01, 0.025, 0.015 BTC)가 들어 있다.

(1) 코인베이스 트랜잭션의 출력 금액을 구하시오.

(2) 이 블록의 머클 트리 리프 개수와, 한 트랜잭션의 머클 증명에 필요한 형제 해시 개수를 쓰시오.

(3) 수수료 0.025 BTC짜리 트랜잭션은 A의 UTXO 0.6 BTC와 0.45 BTC를 입력으로 써서 B에게 0.8 BTC를 보냈다. A가 받는 거스름돈은?

(4) 코인베이스 트랜잭션의 입력 필드가 일반 트랜잭션과 다른 점을 쓰시오.

94. [비트코인] nBits = 0x1c0ae493이다.

(1) E(10진수)와 C를 구하고, 32바이트 타겟 T를 16진수로 쓰시오. (0의 바이트 수로 표현해도 됨)

(2) 다음 블록 해시 중 유효한 것을 모두 고르시오.

h_1 = 0x000000000ae4920f… h_2 = 0x000000000ae4940000… h_3 = 0x0000000000ffff…

(3) 채굴자가 조건을 만족하는 해시를 찾기 위해 반복해서 바꾸는 헤더 필드는?

95. [합의] PBFT로 운영하는 허가형 블록체인을 설계한다.

(1) 노드가 4개일 때 견딜 수 있는 악의적 노드 수와 Commit에 필요한 일치 메시지 수를 쓰시오.

(2) 악의적 노드를 5개까지 견디려면 노드가 최소 몇 개 필요한가?

(3) Prepare 단계에서는 각 노드가 다른 모든 노드에게 메시지를 보낸다. 노드가 4개일 때와 100개일 때 Prepare 메시지 수($n(n-1)$)를 비교하고, PBFT가 개방형 대규모 네트워크에 부적합한 이유를 쓰시오.

96. [이더리움] Alice(EOA)가 컨트랙트의 함수를 호출하는 Type-2 트랜잭션을 보낸다. gas limit 60,000, Max Fee 20 gwei, Priority Fee 2 gwei, 현재 Base Fee 12 gwei.

(1) 이 트랜잭션의 to와 data 필드에는 각각 무엇이 들어가는가?

(2) 실제 사용 가스가 45,000일 때 총 수수료, 소각량, 검증자 수령액을 구하시오.

(3) Alice가 최대로 지불할 수 있었던 금액(gas limit × Max Fee)과 비교해 지불되지 않고 남는 금액은?

(4) 함수가 무한 루프에 빠져 가스를 모두 소진했다면 수수료와 상태는 어떻게 되는가?

97. [이더리움 기초] 이더리움이 비트코인의 한계를 어떻게 넘어서려 했는지 표를 완성하시오.

비트코인의 문제	이더리움의 대응
스크립트가 단순·튜링 불완전 (송금 위주)	(1)
튜링 완전하면 무한 루프 위험 (정지 문제)	(2)
노드마다 하드웨어·OS가 다름	(3)
UTXO 모델은 잔액 조회·프로그래밍이 불편	(4)

98. [이더리움] 블록 탐색기에 다음 블록 정보가 표시되어 있다.

Slot: 9,000,000 / Gas Used: 15,000,000 / Gas Limit: 30,000,000 / Base Fee Per Gas: 10 gwei / Nonce: 0

(1) 이 블록이 속한 에포크 번호를 구하시오. (에포크 0부터 시작)

(2) 블록 가스 사용률(%)과 소각된 ETH(Burnt Fees)를 구하시오.

(3) Nonce가 0인 이유는?

(4) 이 블록이 '확정(Finalized)'되려면 어떤 프로토콜에서 무엇이 필요한가?

99. [솔리디티] SavingWallet 과제 템플릿의 (가)~(사)를 완성하시오.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

contract SavingWallet{
    // (가) owner 상태 변수: 외부에서 읽기 가능, 이더를 받을 수 있어야 함
    // (나) goal 상태 변수: uint, 외부에서 직접 볼 수 없게

    constructor(uint _goal) {
        // (다) owner 설정, ETH 단위 _goal을 wei로 바꿔 저장
    }

    receive() external payable { }

    function isReachGoal() internal view returns (bool) {
        // (라) 잔고가 goal 이상이면 true
    }

    function checkRemain() public view returns (int) {
        // (마) 남은 금액 = goal - 잔고, 이미 넘었으면 0
    }

    function withdraw() public {
        // (바) owner가 아니면 종료(if 사용), 목표 미달이면 종료,
        //      잔액 전부를 owner에게 전송
    }

    function changeOwner(address newOwner) public {
        // (사) owner만 호출 가능, owner를 newOwner로 변경
    }
}
```

100. [솔리디티] 앞 문제의 SavingWallet을 Alice가 goal = 2로 배포했다. 다음을 순서대로 실행할 때 각 단계 직후의 **컨트랙트 잔고**와 **checkRemain() 반환값**(wei)을 쓰시오.
① Bob이 컨트랙트로 1.5 ETH 전송 ② Bob이 withdraw() 호출 ③ Alice가 withdraw() 호출 ④ Carol이 1 ETH 전송 ⑤ Alice가 changeOwner(Bob 주소) 호출 ⑥ Alice가 withdraw() 호출 ⑦ Bob이 withdraw() 호출
또한 ①에서 Bob의 전송을 받아 주는 함수는 무엇인가?

정답 및 해설

Ⅰ. O/X

1. O	**2.** O	**3.** X	**4.** O	**5.** X
6. X	**7.** O	**8.** O	**9.** X	**10.** O
11. X	**12.** O	**13.** X	**14.** X	**15.** O
16. X	**17.** O	**18.** X	**19.** X	**20.** X
21. X	**22.** X	**23.** O	**24.** X	**25.** X

3. X — 대공황 극복을 위해 **금본위제 폐지 + 양적 완화**

5. X — 암호기술은 썼지만 **중앙 기관이 발행하고 이중 지불을 검증** → 가상화폐로 분류

6. X — 하나의 지갑에서 여러 계정(여러 주소)을 만들 수 있음

9. X — 여러 기관이 권한을 나눠 가짐 → 단점이 **운영 참여자 간 이해 조율 필요**

11. X — BLAKE는 결선 **진출**. SHA-3로 선정된 것은 **Keccak** (BLAKE2 = RFC 7693은 맞음)

13. X — 단일 서명 기준으로는 **더 큼**. 장점은 여러 서명을 한 번에 검증하는 효율

14. X — 2017년 RFC 8032로 표준화 → 검증 기간이 **상대적으로 짧음**, 기존 시스템과 호환성 문제

16. X — 그것은 **안전성(Safety)**. 생존성 = 멈추지 않고 블록 생성·합의가 계속 진행됨

18. X — 참여자가 늘면 메시지가 폭증 → **허가형** 블록체인에 적합

19. X — 네트워크 규칙에 따라 **무작위** 배정. 지분이 클수록 선정 **확률**이 높아질 뿐

20. X — 블록 해시는 80바이트 헤더로 계산. 트랜잭션 카운터는 블록 해시에 포함 X

21. X — 코드만 빌려 쓰고 **저장소는 현재 컨트랙트**의 것을 사용

22. X — **2016블록**마다 새로 계산 (평균 10분에 1블록이 되도록)

24. X — 비교표에서 이더리움의 난이도 조절 주기는 '-'. 2016블록(약 2주)은 비트코인

25. X — 프루닝하는 것은 **풀 노드**. 아카이브 노드는 모든 과거 상태를 보관(12~14TB)

Ⅱ. 4지선다

26. ③ — 상하지 않아 오래 가치 유지 → 사용성·보관 용이성 쪽. 보편성은 ④

27. ④ — 오히려 백서는 비트코인 발행을 금 채굴처럼 CPU 연산과 에너지로 하는 방식이라고 설명

28. ④ — 2009년 **C++** 기반 프로토타입

29. ④ — Peer 캐시는 노드 내부의 네트워크 정보일 뿐 장부 상태 변화가 아님

30. ④ — 중앙 기관 없이 스마트 계약으로 실행 → 보장 주체가 없음

31. ③ — 처리 속도가 느림(낮은 확장성). 높은 TPS는 Private의 특징

32. ④ — 해시 함수는 비밀 정보 없이 누구나 실행하는 공개 함수

33. ① — 단계마다 형제 해시: 해시6의 짝 해시5 → 해시56의 짝 해시78 → 해시5678의 짝 해시1234 ($\log_2 8 = 3$개 + 루트)

34. ② — n = 큰 두 소수의 곱. 소인수분해 시 안전성 파괴

35. ④ — ④는 원문을 모두 공개하는 일반 검증 방식

36. ② — x가 **주어진** 상태에서 같은 해시의 x′ 찾기 = 제2역상. 아무 쌍이나 찾는 것이 충돌

37. ②

38. ③ — 길이가 긴 = 누적 작업량이 많은 시퀀스

39. ① — 부정행위 시 슬래싱

40. ④ — 확률적 선정은 PoS. DPoS는 **투표로 선출**

41. ② — Commit = 일치 메시지 2f+1개 이상 받은 노드가 확정 메시지 전파

42. ④ — 비가역성: 개인키 → 공개키 → 주소는 쉽고 반대는 어려움

43. ② — 블록 생성에 사용된 SW/프로토콜 버전

44. ② — ① Geth ③ 컴파일러(Remix·Hardhat) ④ Faucet. 2026년 기준 ethers.js 다운로드가 5.8배 많음

45. ④ — 수수료는 별도 출력이 아니라 입력 – 출력의 **차액**. 채굴자는 코인베이스로 수령

46. ① — 이후 최장 체인으로 대표 선정 → 보상 → MemPool에서 삭제

47. ① — E = 0x1b = 27 → 뒤 (E–3) = 24바이트, 앞 32 – 27 = 5바이트 (5 + 3 + 24 = 32)

48. ② — ⌊700,000 / 210,000⌋ = 3번 반감 → $50 / 2^3 = 6.25$

49. ②

50. ④ — 결정적 실행이 깨지면 합의가 불가능

51. ② — 비트코인 SegWit 이전 1MB

52. ② — storageRoot = 계약 저장소 MPT의 루트 해시

53. ② — Base Fee(자동 조정·소각)와 Priority Fee(팁)를 분리, Max Fee로 상한 설정

54. ④

55. ② — 내부 타입 = 현재 컨트랙트 안에서 함수를 참조

Ⅲ. 단답형 · 계산

56. **① 1983 ② 1989 ③ 1998 ④ 2009** — DigiCash는 1998년 파산 — B-Money 제안과 같은 해

57. **① 암호 소프트웨어 배포와 이를 통한 개인 프라이버시 보호 ② 암호화 이메일·전자화폐·익명 네트워크 아이디어 공유**

58. **PoW 해시 퍼즐 때문 — 블록 해시가 난이도로 정해진 타겟 값 T보다 작아야 유효하므로, 유효한 블록 해시는 앞자리가 0으로 채워짐** — 넌스를 바꿔 가며 조건을 만족하는 해시를 찾은 결과

59. **① Private ② Consortium ③ Public ④ Hybrid**

60. **① 리프 5개 → 마지막 복사해 6개 → 3개 → 복사해 4개 → 2개 → 루트 1개 ② 3개** — 홀수 개면 마지막을 복사해 짝수로 맞춤. 높이 3이므로 형제 해시 3개(Tx5는 자기 복사본과 짝)

61. **① 2개 ② 3개** — t-of-N: N명 중 t명 이상 모이면 성립 → 5 − 3 = 2개까지 분실 허용

62. **① 무결성 ② 부인 방지 ③ 메시지 출원 인증**

63. **① f = 4 ② 2f + 1 = 9개** — 3f + 1 ≤ 13 → f ≤ 4

64. **체인: A → C → E → F / 스테일 블록: B, D** — D·E 시점에는 동률(Tie) → F가 붙으며 C 가지가 더 길어짐(누적 작업량 최대)

65. **평균 2^{16} = 65,536번 / 검증은 1번** — 답은 찾기 어렵고 검증은 쉬워야 한다는 퍼즐의 원칙

66. **1년 블록 수 = 365 × 24 × 6 = 52,560개 → 52,560 × 80바이트 = 4,204,800바이트 ≈ 4.2MB** — 헤더만 저장하므로 스마트 기기에도 적합

67. **① 오른다 (타겟 T는 작아짐) ② 12/14 = 6/7 ≈ 0.86배** — 기대 시간 2016 × 10분 = 14일보다 빨랐으므로 더 어렵게 조정 (실제 비트코인 규칙: 새 T = 기존 T × 실제 시간/기대 시간)

68. **1,050,000 / 1.5625 BTC** — 840,000 + 210,000. 보조금 50 → 25 → 12.5 → 6.25 → 3.125 → 1.5625

69. **① 0.05 + 0.02 = 0.07 BTC ② A 2.0, B 0.7, B 0.5 ③ A 1.25, C 1.1, B 0.08** — UTXO는 통째로 소비되고 새 UTXO(수신자 + 거스름돈)가 생성

70. **0.075 BTC** — 850,000 → 4번 반감 → 보조금 3.125 BTC. 3.2 − 3.125

71. **① 210,000 × 50 = 10,500,000 BTC ② 50%** — 이후 25% → 12.5% … 등비급수 합 = 2,100만 개

72. **약 60분 / 뒤에 블록이 쌓일수록 그 블록을 뒤집으려면 더 긴 체인(더 많은 작업량)을 만들어야 해서 분기·이중 지불 가능성이 급격히 줄어듦**

73. **① 80,000 × 25 = 2,000,000 gwei = 0.002 ETH ② 없음 ③ 취소(reverted)** — 실패해도 사용된 가스는 환불되지 않음

74. **가스당 25 gwei (Base 22 + 팁 3) ① 525,000 gwei ② 462,000 gwei ③ 63,000 gwei** — Base + Priority = 27 > Max Fee 25 → 상한에 걸려 팁이 25 − 22 = 3 gwei로 줄어듦

75. **① 30 gwei ② 630,000 gwei = 0.00063 ETH** — gasPrice 필드는 Wei 단위 (1 gwei = 10^9 Wei)

76. **다르다 — CA 주소는 배포자 주소 + nonce를 해시해 결정되는데, 첫 배포 후 배포자의 nonce가 증가했기 때문**

77. **슬롯 86,400 / 12 = 7,200개, 에포크 7,200 / 32 = 225개**

78. **public 상태 변수 이름으로 자동 생성된 게터(getter) 함수 / 다른 계약이나 외부 계정이 자유롭게 읽을 수 있게 하려는 의도**

79. **goal = _goal * 1 ether; (또는 _goal * 10**18) → 3 × 10^{18} wei** — 1 ETH = 10^{18} Wei

80. **남은 금액 = goal − 잔고가 음수가 될 수 있기 때문 / uint 빨셈에서 음수가 나오면 0.8 이상에서는 언더플로로 트랜잭션이 되돌려짐(revert)** — 그래서 int(goal) − int(잔고)로 계산하고 0 이하이면 0을 반환

Ⅳ. 단순기술형

81. **신자유주의 풍조 속 다국적 기업이 경쟁을 위해 정보를 대량 수집하고 권력기관과 유착, 인터넷 상거래·SNS·빅데이터 확산으로 개인정보가 중앙 서버에 저장됨 → 거래 내역·위치·소비 습관이 추적되는 감시·검열 수단이 됨. → 암호로 프라이버시를 지키려는 사이퍼펑크 운동 → 중앙 기관 없이 암호기술로 신뢰를 만드는 비트코인으로 이어짐.**

82. **트랜잭션 수정 → 머클 루트가 바뀜 → 헤더가 바뀌어 블록 해시가 바뀜 → 다음 블록 헤더의 '이전 블록 해시'와 불일치 → 이후 모든 블록을 다시 만들어야(PoW도 다시) 함. 또한 다른 노드들이 가진 분산 장부와도 달라 바로 탐지됨.**

83. 공격자가 파일과 해시 값을 함께 바꿔치기하면 해시가 일치해도 위조 파일 — 해시는 누구나 계산하는 공개 함수라 '누가 만든 값인지'를 보장하지 못함(신뢰 사이트에서 받은 해시여야 의미). 전자서명은 개발자가 개인키로 해시에 서명 → 공개키로 검증하면 무결성과 함께 출처(메시지 출원 인증)까지 확인.

84. 메시지 전체에 서명하면 느리고 서명이 커짐 → 고정 길이 해시에 서명해 효율화. 충돌 저항성이 깨져 h(m) = h(m′)인 m, m′을 만들 수 있으면, m에 받은 서명이 m′에도 그대로 유효 → 서명 위조(예: 정상 계약서로 서명받고 위조 계약서에 사용).

85. 정직하게 블록을 만들면 블록 보조금 + 수수료 보상을 받음. 부정행위(잘못된 블록)는 많은 연산(전력) 비용을 들이고도 검증이 쉬워 바로 거부됨 → 비용만 날림. 따라서 경제적으로 정직이 이득.

86. 더 긴 체인이 나타나면 기존 블록이 스테일 블록이 되어 뒤집힐 수 있음. 다만 블록이 뒤에 쌓일수록 뒤집힐 확률이 급격히 작아짐 → 확률적. PBFT는 2/3 이상(2f+1) 동의 시 즉시·결정적으로 확정되어 되돌아가지 않음.

87. SPV는 트랜잭션 원문이 없으므로 풀 노드에 해당 트랜잭션과 머클 경로를 요청 → 트랜잭션 해시와 형제 해시들을 차례로 해시해 계산한 값이 자신이 가진 헤더의 머클 루트와 같으면 포함된 것. 경로 크기가 O(log N)이라 적은 데이터로 검증 가능.

88. ① 이전 11개 블록 타임스탬프의 중간값보다 커야 함 → 과거로 시간을 되돌린 블록 방지 ② 네트워크 시간보다 2시간 이상 미래면 안 됨 → 미래 시간을 적어 (난이도 계산 등을) 조작하는 블록 방지.

89. 채굴자는 수수료가 높은 트랜잭션을 우선 선택(빨리 처리하려면 높은 수수료가 유리). 보조금은 210,000블록마다 절반이 되어 결국 0에 가까워짐 → 채굴 보상의 대부분을 수수료가 차지하게 되어, 수수료가 채굴자의 참여 유인이자 네트워크 보안의 원천이 됨.

90. 장점: 배포 이후 누구도 계약 내용을 마음대로 바꿀 수 없어 제3자 없이도 신뢰하고 거래 가능. 단점: 코드에 버그·취약점이 있어도 고칠 수 없어 보안 사고로 이어질 수 있음(DeFi의 스마트 계약 버그 위험).

91. codeHash = EVM 바이트코드의 keccak256 해시 — 코드가 배포 후 불변이므로 해시도 불변. storageRoot = 계약 저장소(상태 변수) MPT의 루트 해시 — 트랜잭션이 상태 변수를 바꿀 때마다 저장소가 바뀌어 루트도 갱신.

92. ① Remix VM: 브라우저 안의 가상 네트워크, 실제 체인 X / Sepolia: 실제 이더리움과 유사한 공개 테스트넷 ② Remix VM은 제공되는 테스트 계정으로 바로 배포 / Sepolia는 MetaMask 지갑 연결(Browser extension → Sepolia Testnet - MetaMask) + Faucet으로 받은 SepoliaETH로 가스 지불 ③ Remix VM은 Remix 로그로만 / Sepolia는 sepolia.etherscan.io에서 실제 스캐너처럼 확인.

Ⅴ. 복합기술형

93. (1) ⌊880,000/210,000⌋ = 4번 반감 → 보조금 3.125 BTC + 수수료 0.05 = 3.175 BTC (2) 코인베이스 포함 4개(짝수라 복사 불필요), 형제 해시 $\log_2 4$ = 2개 (3) 1.05 − 0.8 − 0.025 = 0.225 BTC (4) 이전 트랜잭션(UTXO)을 참조하지 않음, ScriptSig에 임의 데이터(채굴자 메시지) 작성 가능.

94. (1) E = 0x1c = 28, C = 0x0ae493 → 앞 0 4바이트 + 0ae493 + 뒤 0 25바이트 = 0x00000000 0ae493 000…000 (2) h_1, h_3 (h_1은 0ae492 < 0ae493, h_3은 앞자리 0이 더 많아 더 작음, h_2는 0ae494 > 0ae493이라 무효) (3) 넌스(Nonce)

95. (1) f = 1, 2f + 1 = 3개 (2) 3 × 5 + 1 = 16개 (3) 4 × 3 = 12개 vs 100 × 99 = 9,900개 — 노드 수의 제곱에 비례해 통신량이 폭증 → 참여자가 많은 개방형 환경에서는 성능 저하, 참여자를 관리할 수 있는 허가형에 적합.

96. (1) to = 컨트랙트(CA) 주소, data = 호출할 함수와 인자 값 (2) 가스당 12 + 2 = 14 gwei → 총 630,000 gwei(0.00063 ETH), 소각 45,000 × 12 = 540,000 gwei, 검증자 45,000 × 2 = 90,000 gwei (3) 60,000 × 20 = 1,200,000 gwei − 630,000 = 570,000 gwei (반환) (4) 60,000 × 14 = 840,000 gwei 지불, 상태 변경 취소(reverted), 가스 환불 없음.

97. (1) 튜링 완전한 스마트 컨트랙트 + 고급 언어 Solidity로 범용 프로그램 실행 (2) 가스: 연산량에 수수료를 매기고 gas limit으로 상한 → 소진 시 reverted (3) EVM: 모든 노드에서 같은 입력 → 같은 출력을 보장하는 가상 실행 환경 (4) 계정 모델: balance로 잔액을 바로 갱신, 계정 간 조회·송금이 편리, 이중 지불 해결이 쉬움.

98. (1) 9,000,000 / 32 = 281,250 → 에포크 281,250 (2) 50% / 15,000,000 × 10 gwei = 150,000,000 gwei = 0.15 ETH (3) PoS 전환 이후 채굴(넌스 탐색)이 없어 0 (4) Casper FFG — 스테이킹 지분 2/3 이상의 투표.

99.

```
(가) address payable public owner;
(나) uint private goal;
(다) owner = payable(msg.sender);  goal = _goal * 1 ether;
(라) return address(this).balance >= goal;
(마) if (isReachGoal()) return 0;
     return int(goal) - int(address(this).balance);
(바) if (msg.sender != owner) return;
     if (!isReachGoal()) return;
     owner.transfer(address(this).balance);
(사) if (msg.sender != owner) return;
     owner = payable(newOwner);
```

owner는 송금받아야 하므로 address payable (transfer는 payable 주소에서만 가능). newOwner는 address라 그대로 대입하면 컴파일 오류 → payable(newOwner). ※ transfer는 0.8.31부터 deprecated — call로도 구현 가능

100. ① 1.5 ETH / 5×10^{17} ② 1.5 / 5×10^{17} (owner 아님 → 종료) ③ 1.5 / 5×10^{17} (목표 미달 → 종료) ④ 2.5 / 0 ⑤ 2.5 / 0 (owner = Bob) ⑥ 2.5 / 0 (Alice는 더 이상 owner 아님) ⑦ 0 / 2×10^{18} (Bob에게 2.5 ETH 전송) — 받아 주는 함수: receive() external payable. ※ 로컬 테스트 체인에서 실제로 실행해 확인한 결과